

COMPILERS QUIZ 4

We are going to extend the interpreter with functions and named globals.

1 Symbols

A symbol is a new type of data. It can be any sequence of characters without whitespace. For lexing, we identify symbols with a quote as a prefix. Two symbols are equal iff they are the same sequence of characters.

This program:

```
'hello 'world 'good-bye
```

pushes three symbols into the stack and quits.

Here are some operations on symbols:

```
`is-symbol?' true iff top of stack is a symbol.
```

```
`sym=' true iff top two values are the same symbol.
```

2 Defining functions

The ``P 'sym def'` word associates symbols to procedures. Once a symbol is associated, it can be used as a word (without quote) to execute the procedure. As usual, ``def'` removes ``P'` and ``'sym'` from stack.

```
{ { dup 0 = } { pop 1 } { dup dec fact * } if } 'fact def
```

```
5 fact print
```

prints 120.

All symbols are global. The program:

```
{ x y + print } 'foo def
```

```
{ { 1 } 'x def { 2 } 'y def foo } 'bar def
```

```
bar
```

should print 3 and the stack should be empty at the end of this program. Observe that the ``x'` and ``y'` defined by executing ``bar'` can be seen by ``foo'`.